

# Acesso a Bases de Dados usando JDBC

Eduardo Piveta

# Acesso a bases de dados

- Existem diferentes formas de acessar e manipular dados em programas OO em Java. As mais comuns são:
  - Via JDBC
  - Via mapeamento objeto-relacional (eg.: hibernate)
  - Via persistência gerenciada pelo container.
- Nesta aula veremos como manipular dados em bases de dados usando JDBC.

# BD

- Uma **base de dados** é composta normalmente de um conjunto de **tabelas**.
- Uma **tabela** contém registros de um determinado tipo. Exemplo de tabelas: **clientes, funcionarios, cargos, salarios** etc.
- Cada **tabela** pode possuir diversos **campos**. Por exemplo, a tabela cliente pode ter: **codigo, nome, cpf, endereco**, etc.
- Vocês devem estar vendo algo disso em BD...

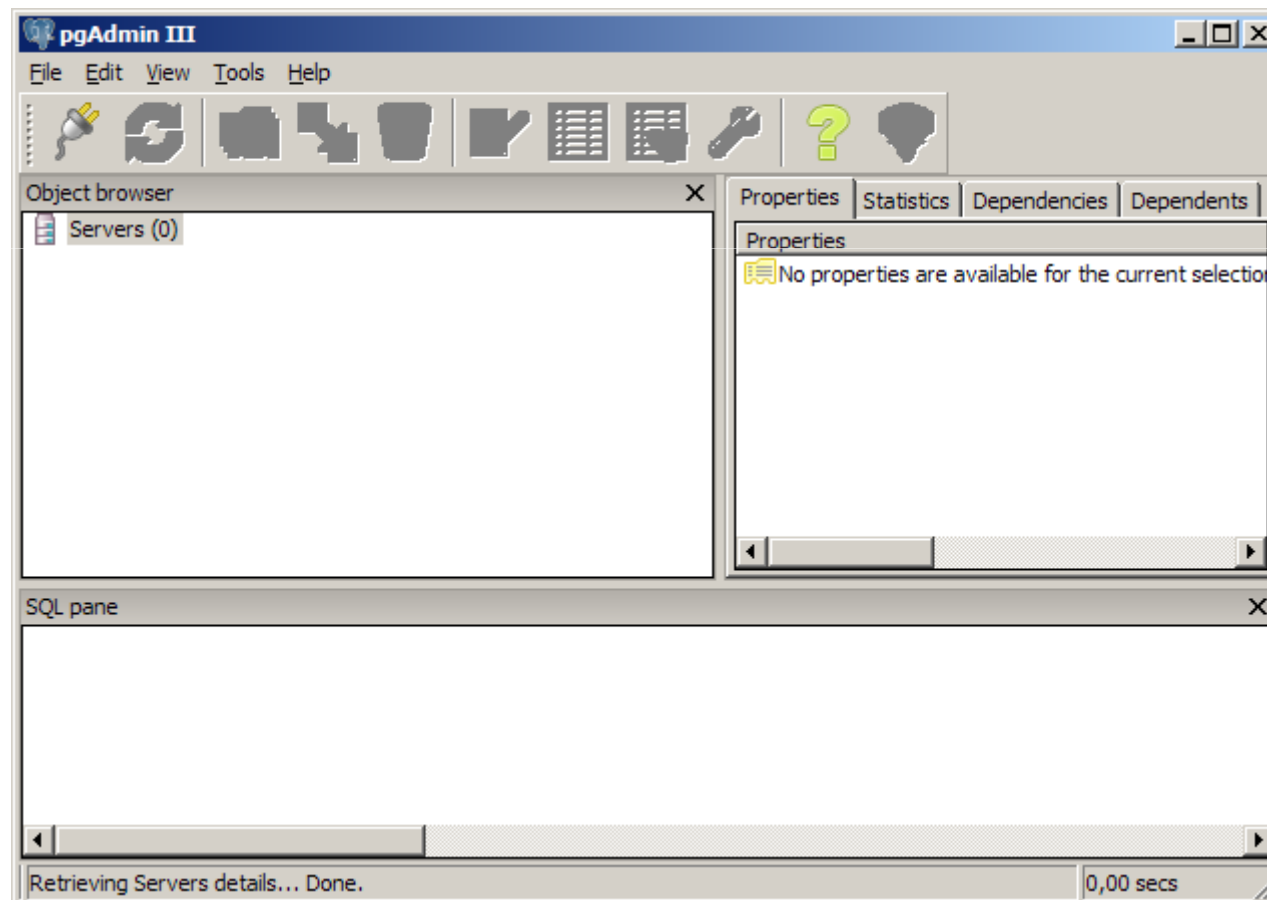
# Criação de uma base de dados no PostgreSQL

- Para os exemplos de acesso a BD, estaremos usando o postgres:  
<http://www.postgresql.org/>
- É provavelmente o melhor BD gratuito.
- Criaremos inicialmente uma **base de dados** chamada **livraria** com UMA **tabela** chamada **clientes**.

# Passo 1: Conectando ao servidor

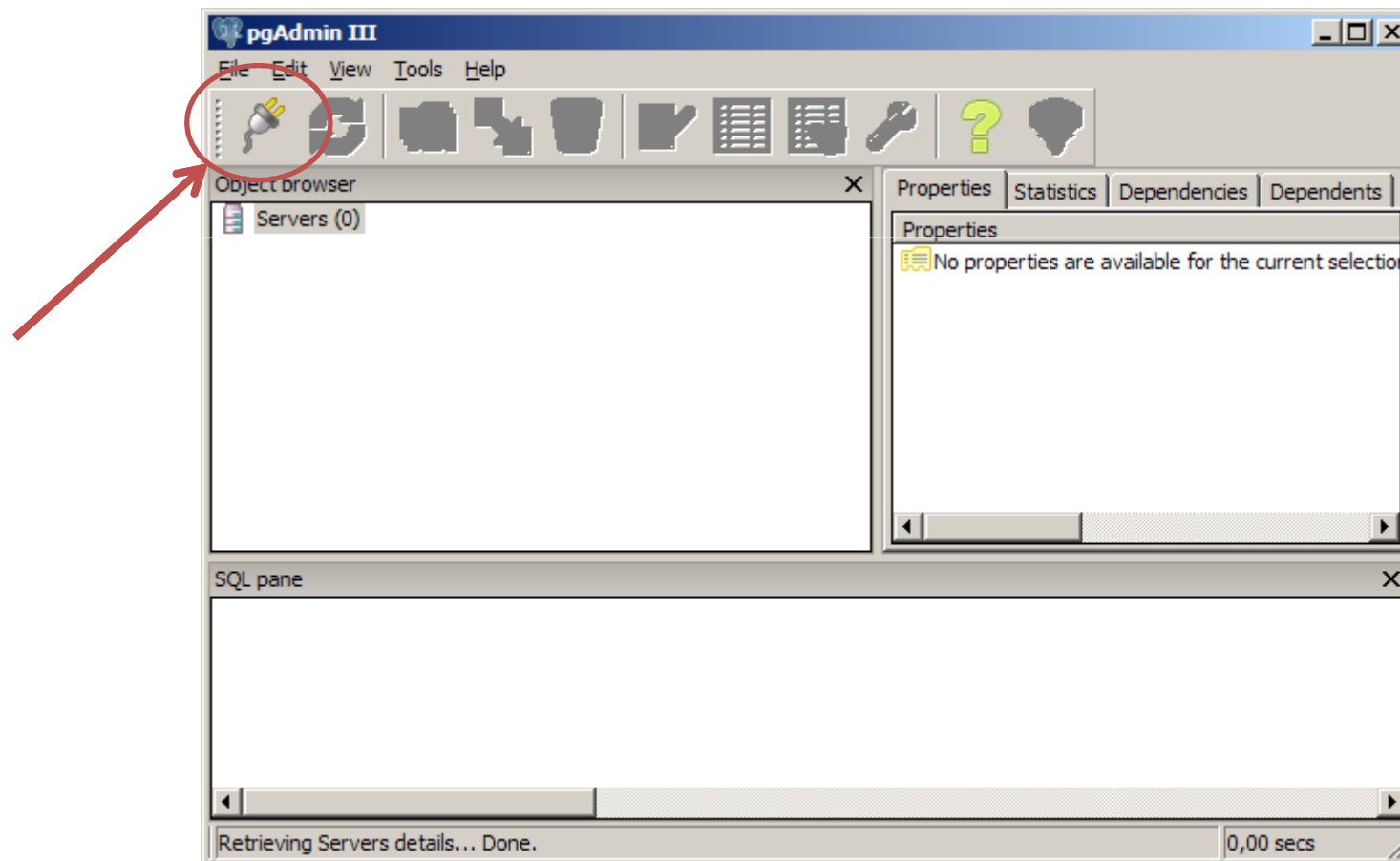
# Acessando o postgres

- Em Programas > PostgreSQL > pgAdmin III...



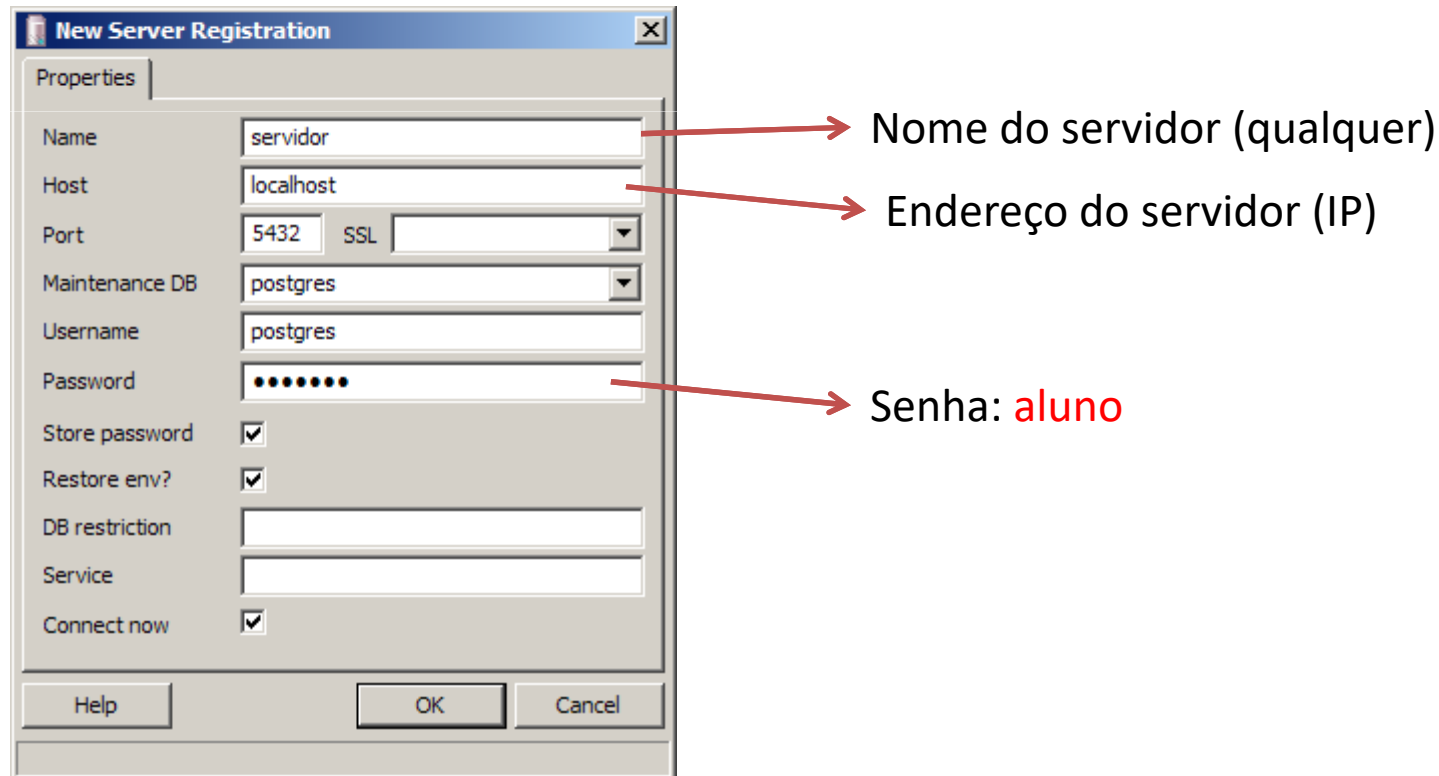
# Conectando a um servidor

- Nos conectaremos no servidor local...



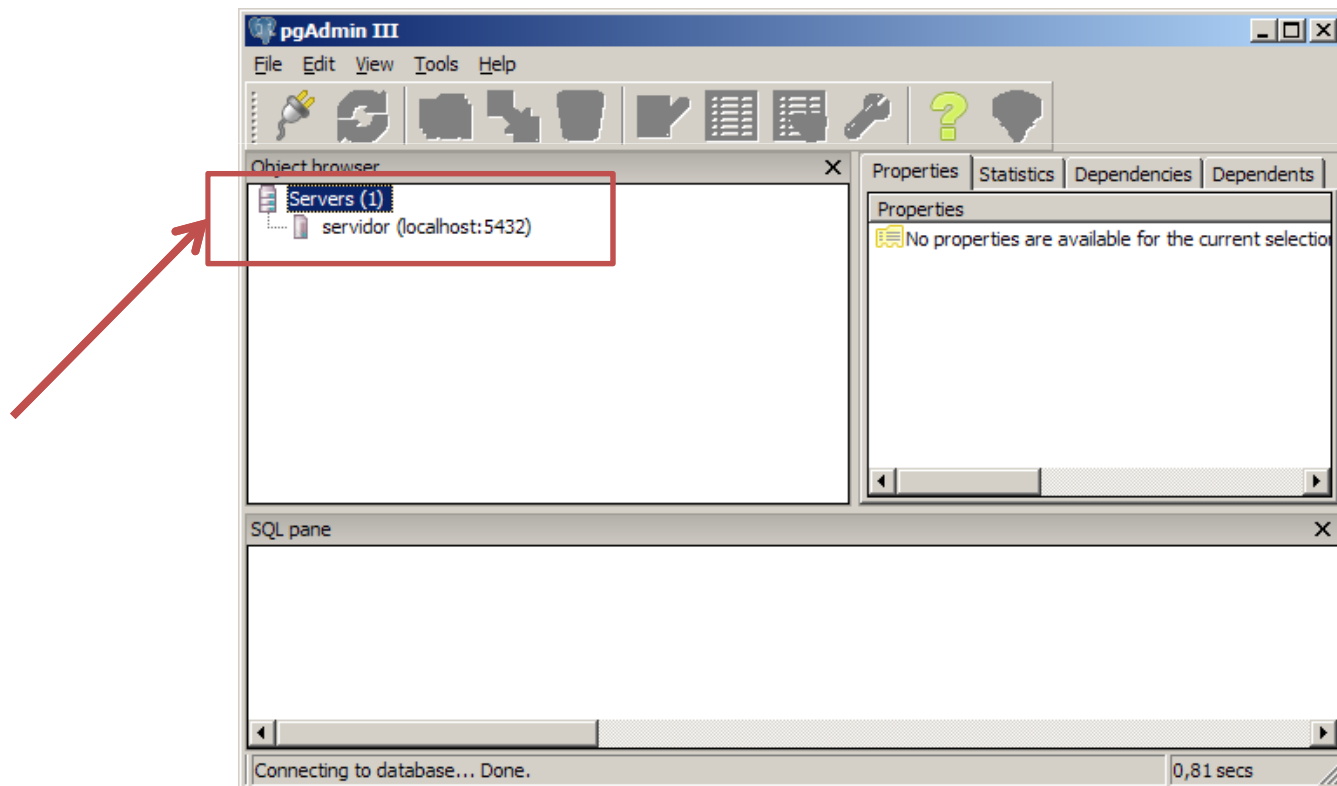
# Conectando ao servidor

- Ou através do menu:
  - File > Add Server ... (senha: **aluno**)



# Conectado no servidor

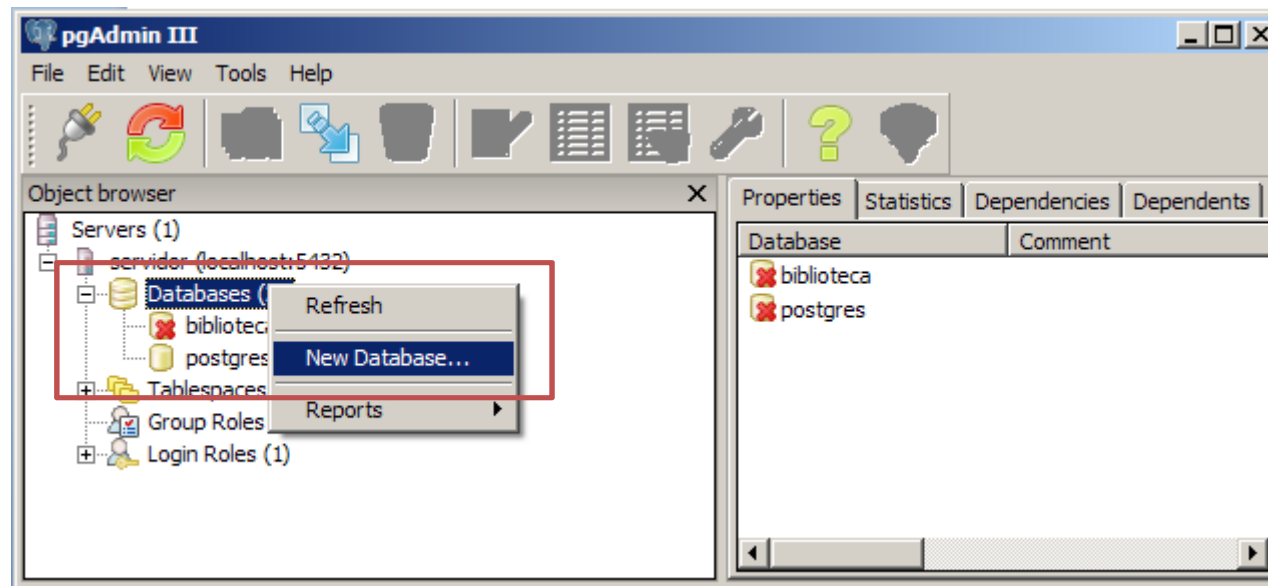
- Agora (se tudo deu certo) você está conectado no servidor local.



## Passo 2: Criando a Base de Dados

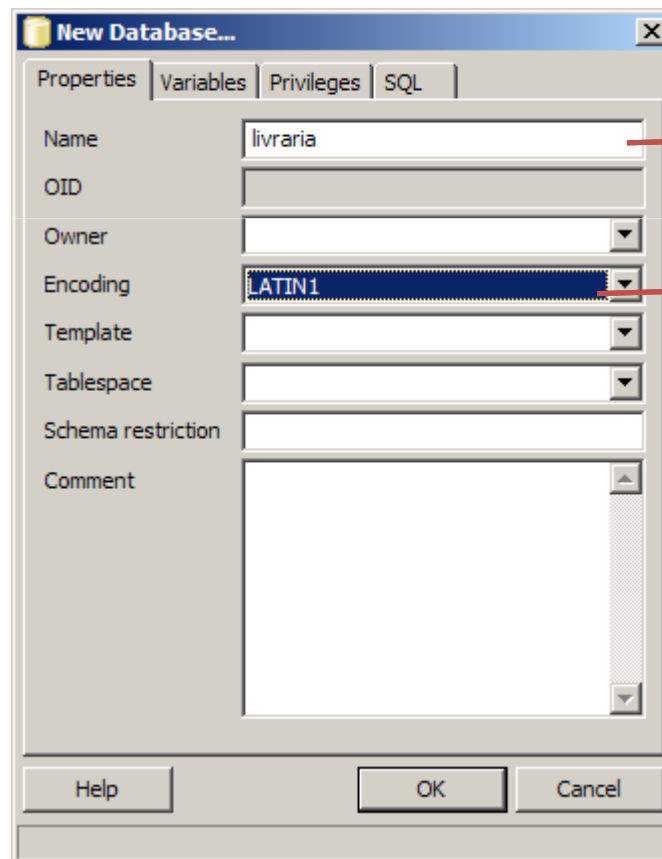
# Criando a base de dados

1. Clique no servidor..
2. Vá em “Databases” e clique em “New Database...”



# Criando a base de dados

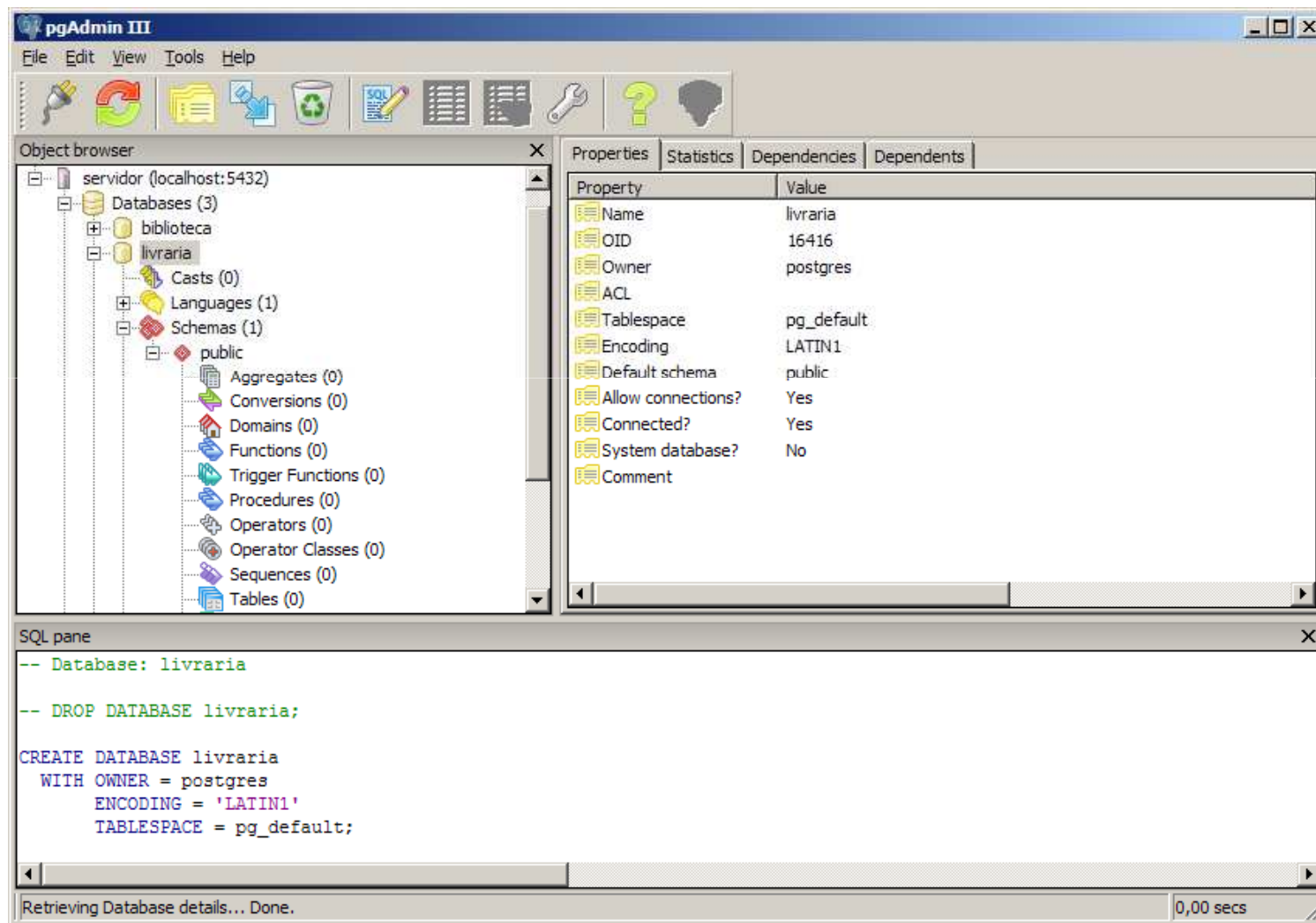
- O principal é o nome da base de dados



Nome da base de dados

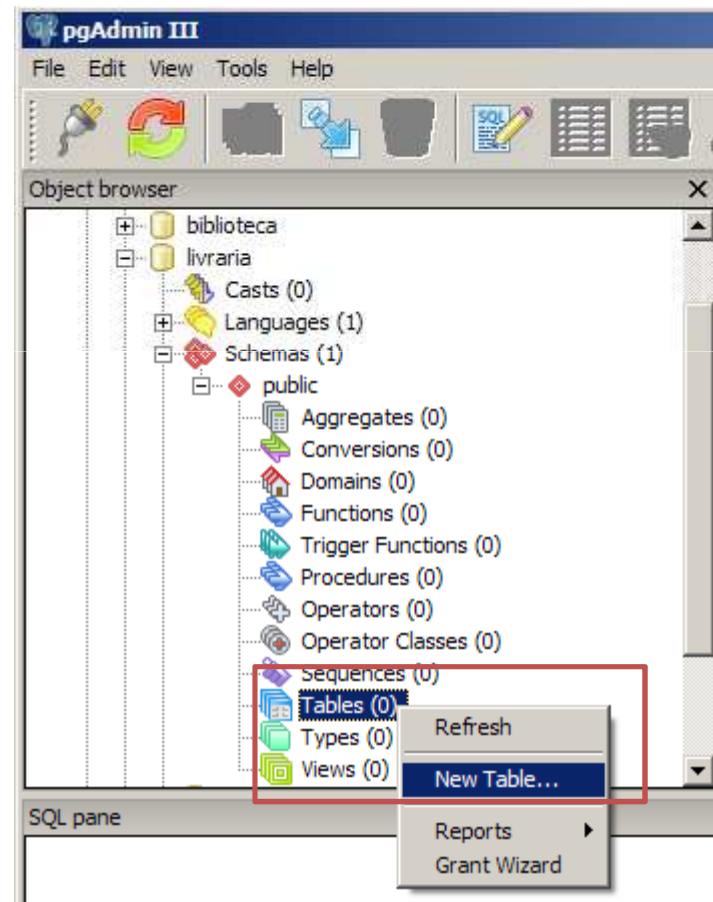
Codificação de caracteres

# Base de Dados Criada



Passo 3: Criando a tabela **clientes**

# Criando a tabela clientes



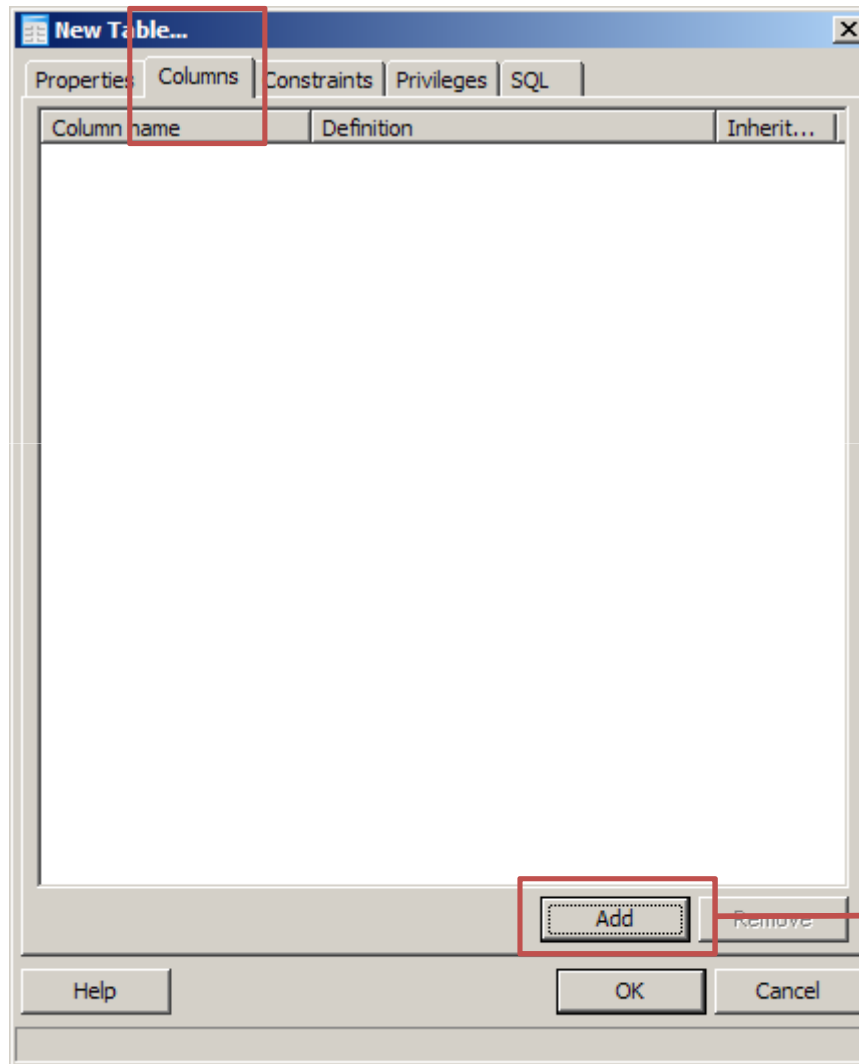
# Passo 1 - Propriedades

The image shows a 'New Table...' dialog box with the following fields and controls:

- Name:** A text field containing 'clientes', highlighted with a red box.
- OID:** A text field.
- Owner:** A dropdown menu.
- Tablespace:** A dropdown menu.
- Has OIDs:** A checkbox.
- Inherits from tables:** A section containing a list box, a 'Remove' button, and an 'Add' button.
- Comment:** A large text area.
- Use replication:** A dropdown menu.
- Buttons:** 'Help', 'OK', and 'Cancel' at the bottom.

Nome da tabela

## Passo 2 - Colunas



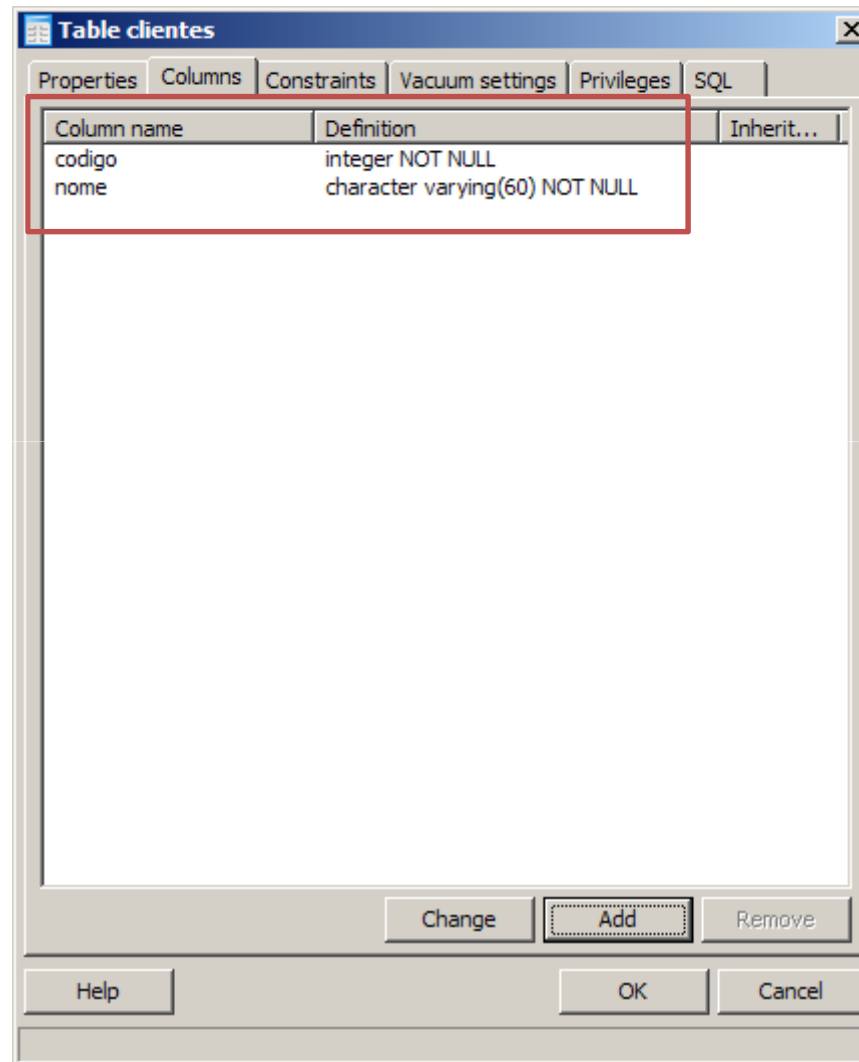
Adicionar nova coluna

## Passo 2 – Colunas > Adicionando codigo e nome

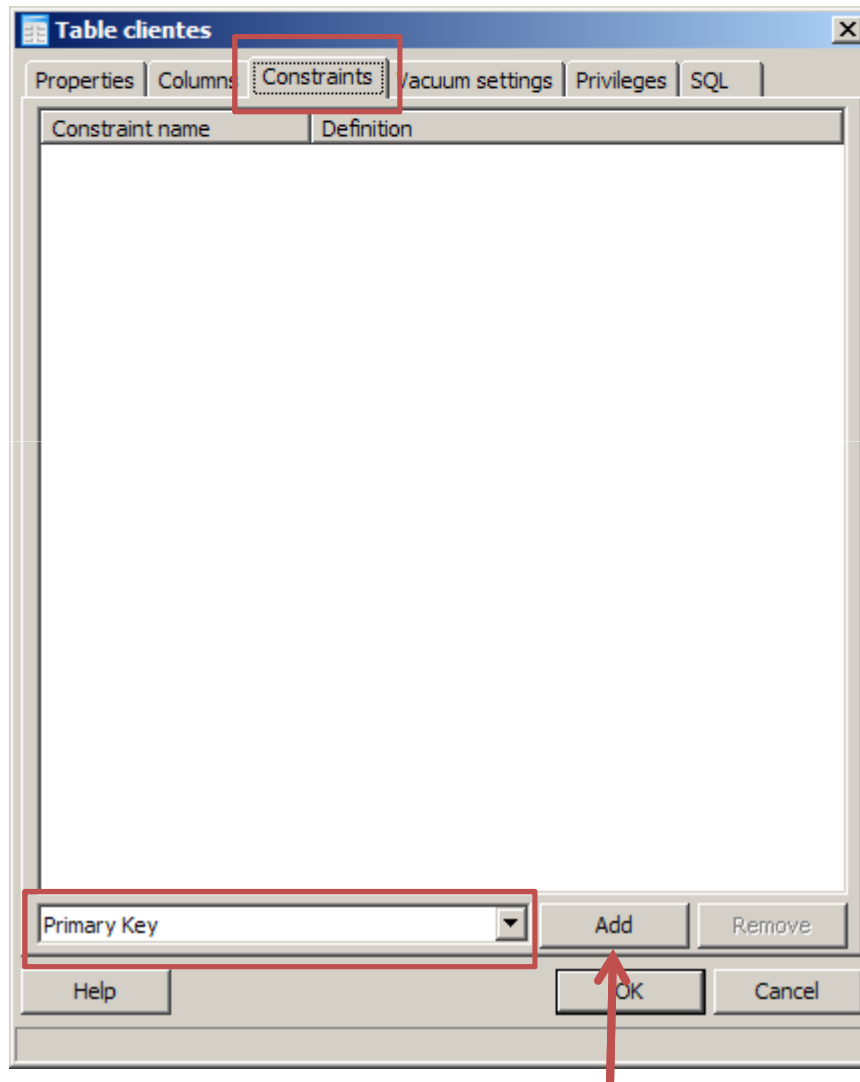
The 'New Column...' dialog box is shown with the 'Properties' tab selected. The 'Name' field contains 'codigo' and the 'Data type' dropdown is set to 'integer'. Both fields are highlighted with a red rectangle. The 'Not NULL' checkbox is checked and also highlighted with a red rectangle. The 'Length' and 'Precision' fields are empty. The 'Default value' field contains '<new sequence>'. The 'Sequence' dropdown is set to '<new sequence>'. The 'Statistics' and 'Comment' fields are empty. The 'Use replication' dropdown is set to the default. At the bottom are 'Help', 'OK', and 'Cancel' buttons.

The 'New Column...' dialog box is shown with the 'Properties' tab selected. The 'Name' field contains 'nome' and the 'Data type' dropdown is set to 'character varying'. Both fields are highlighted with a red rectangle. The 'Length' field contains '60' and the 'Precision' field is empty. The 'Not NULL' checkbox is checked and also highlighted with a red rectangle. The 'Default value' field contains '<new sequence>'. The 'Sequence' dropdown is set to '<new sequence>'. The 'Statistics' and 'Comment' fields are empty. The 'Use replication' dropdown is set to the default. At the bottom are 'Help', 'OK', and 'Cancel' buttons.

## Passo 2 – Colunas adicionadas

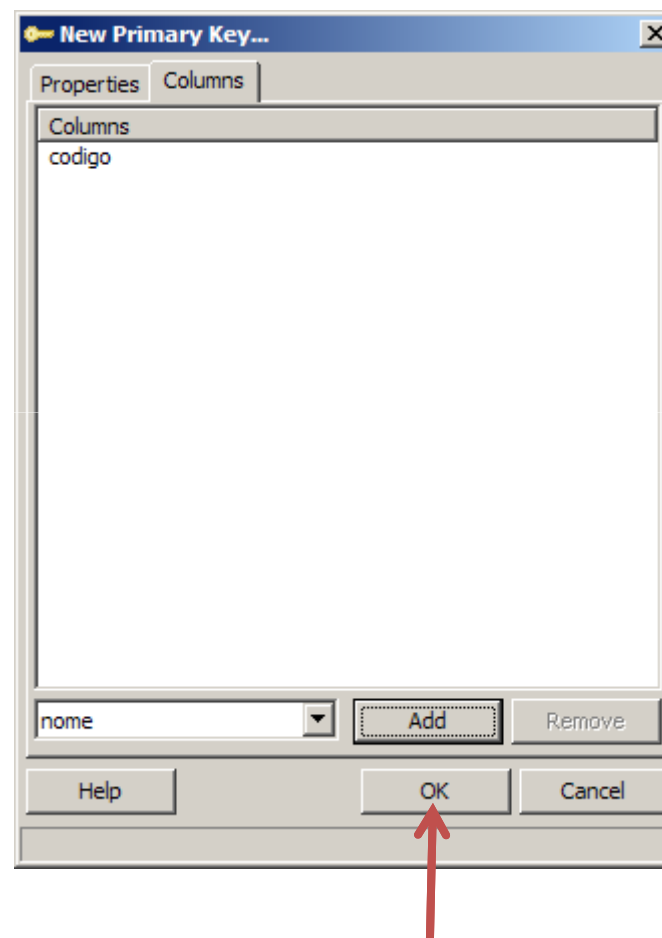
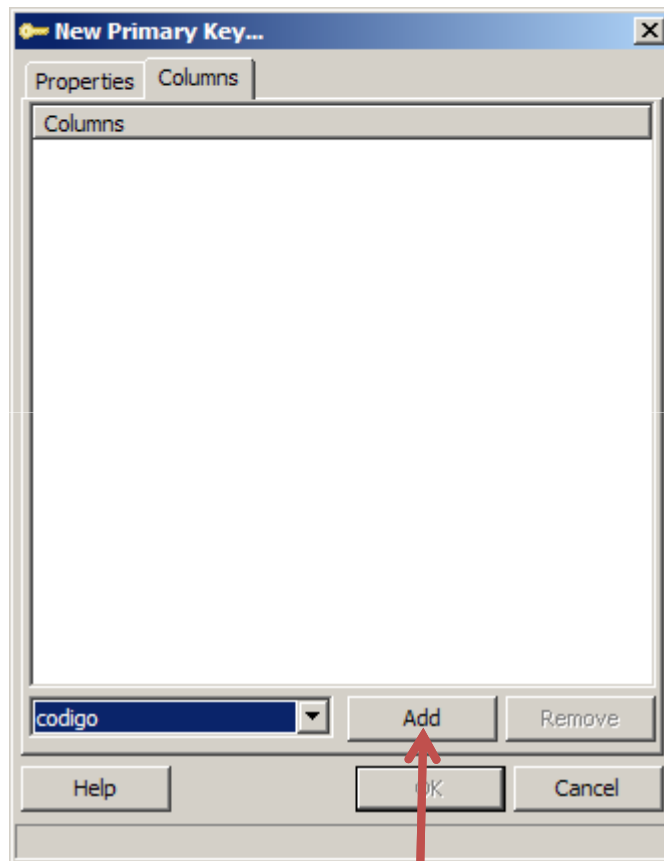


## Passo 3 – Restrições – Chave Primária

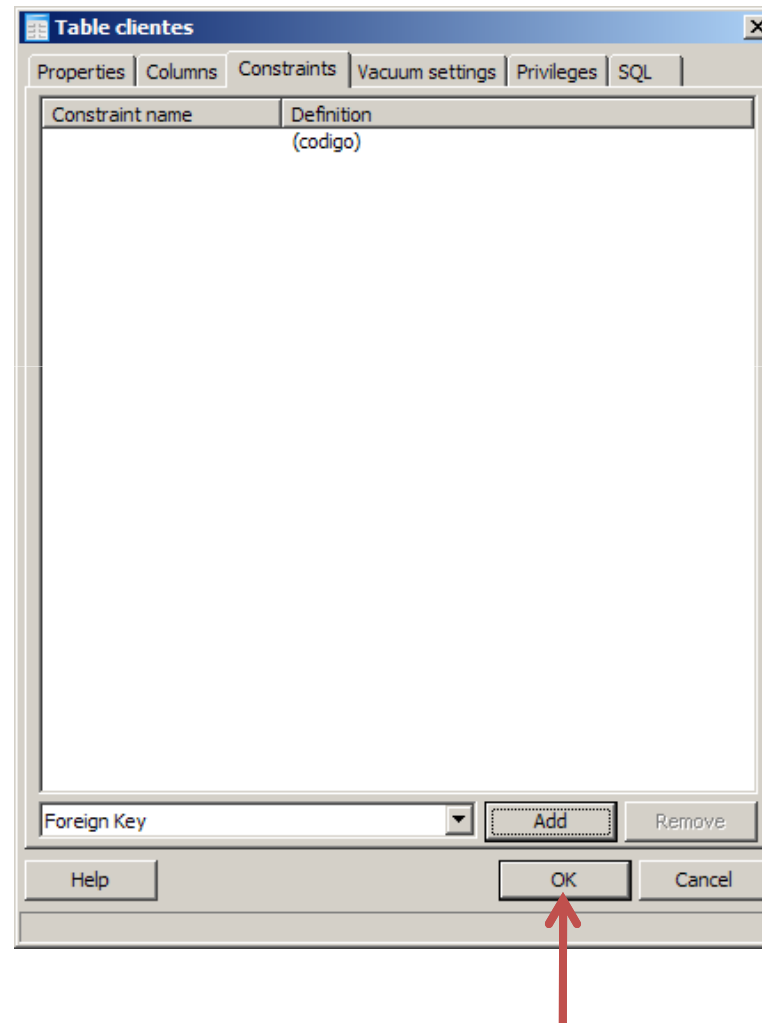


A chave primária de uma tabela é composta pelo conjunto de campos cujos valores identificam unicamente um registro.

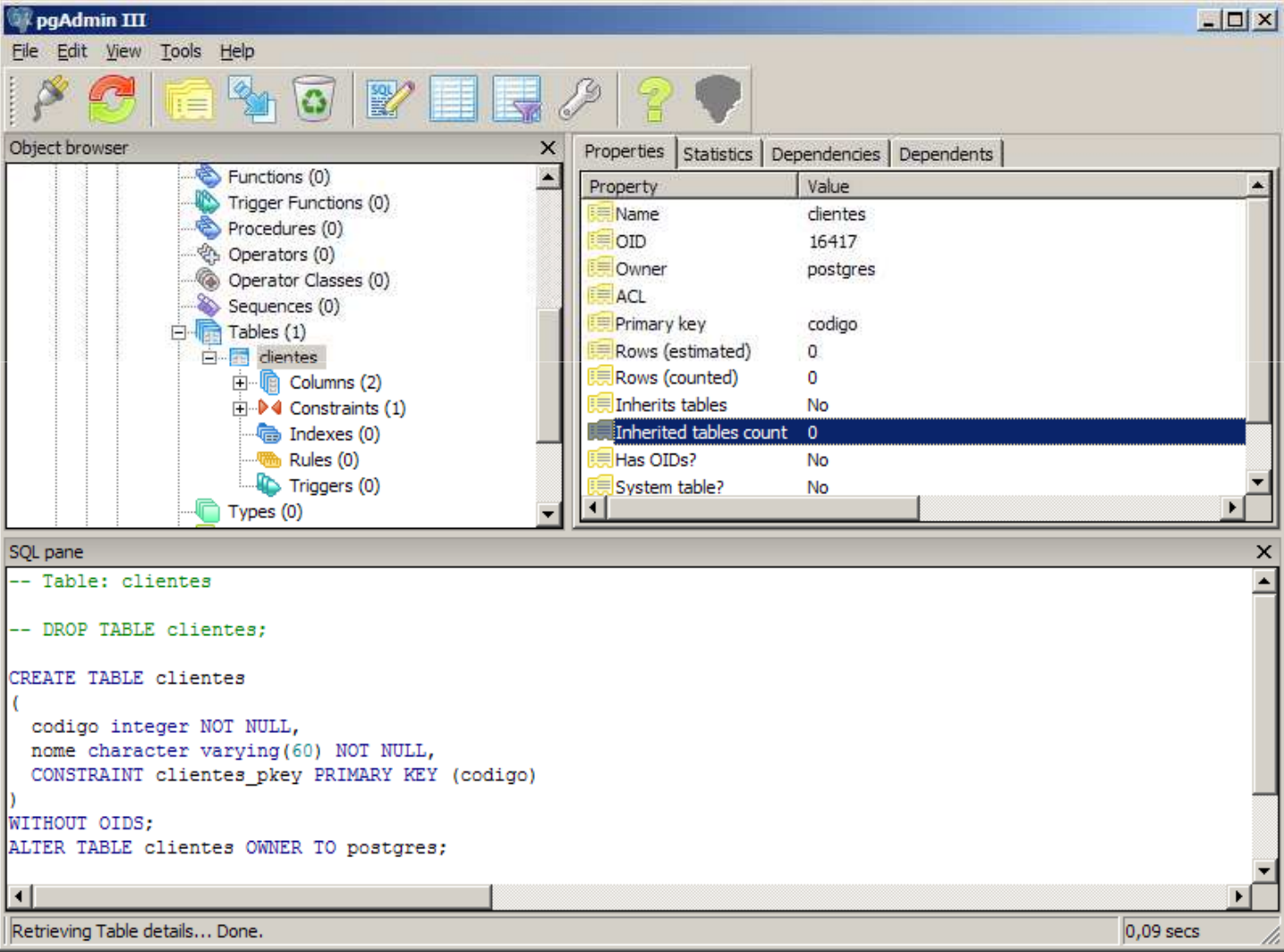
## Passo 3 – Restrições – Chave Primária



## Passo 4 - Finalizando a criação da tabela



# Tabela criada



The screenshot displays the pgAdmin III interface. On the left, the 'Object browser' shows the database structure with 'Tables (1)' expanded, listing the 'clientes' table. The 'Properties' tab on the right shows details for the 'clientes' table, including its name, OID, owner, and primary key. The 'SQL pane' at the bottom contains the SQL script used to create the table. A red arrow points from the text 'SQL Gerado' to the SQL pane.

Object browser

- Functions (0)
- Trigger Functions (0)
- Procedures (0)
- Operators (0)
- Operator Classes (0)
- Sequences (0)
- Tables (1)
  - clientes
    - Columns (2)
    - Constraints (1)
    - Indexes (0)
    - Rules (0)
    - Triggers (0)
- Types (0)

Properties

| Property               | Value    |
|------------------------|----------|
| Name                   | clientes |
| OID                    | 16417    |
| Owner                  | postgres |
| ACL                    |          |
| Primary key            | codigo   |
| Rows (estimated)       | 0        |
| Rows (counted)         | 0        |
| Inherits tables        | No       |
| Inherited tables count | 0        |
| Has OIDs?              | No       |
| System table?          | No       |

SQL pane

```
-- Table: clientes
-- DROP TABLE clientes;

CREATE TABLE clientes
(
    codigo integer NOT NULL,
    nome character varying(60) NOT NULL,
    CONSTRAINT clientes_pkey PRIMARY KEY (codigo)
)
WITHOUT OIDS;
ALTER TABLE clientes OWNER TO postgres;
```

Retrieving Table details... Done. 0,09 secs

SQL  
Gerado



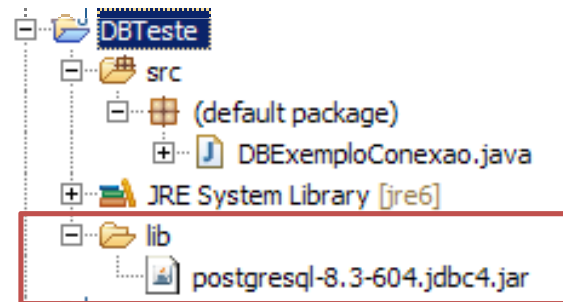
## Passo 4: Acessando a base de dados via JDBC

# Para acessar um BD é preciso...

- Um driver JDBC faz com que as consultas possam ser executadas nativamente no BD.
- Baixem o driver do site do fabricante
  - Para o postgres:
    - <https://jdbc.postgresql.org/>
  - Para o mysql:
    - <https://dev.mysql.com/downloads/connector/j/>

# Adicionando o driver no projeto

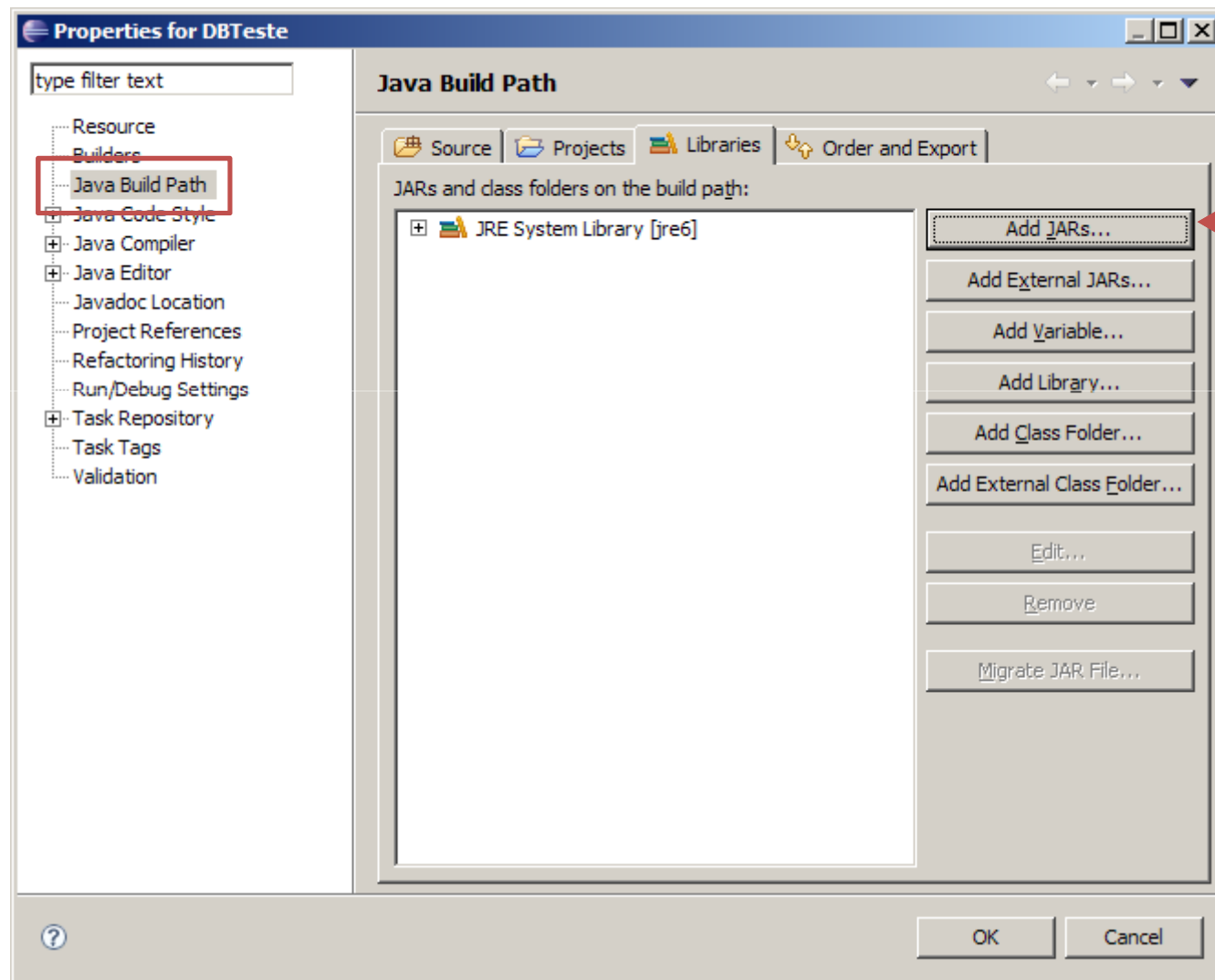
- Coloque o .jar do driver em um diretório do projeto.



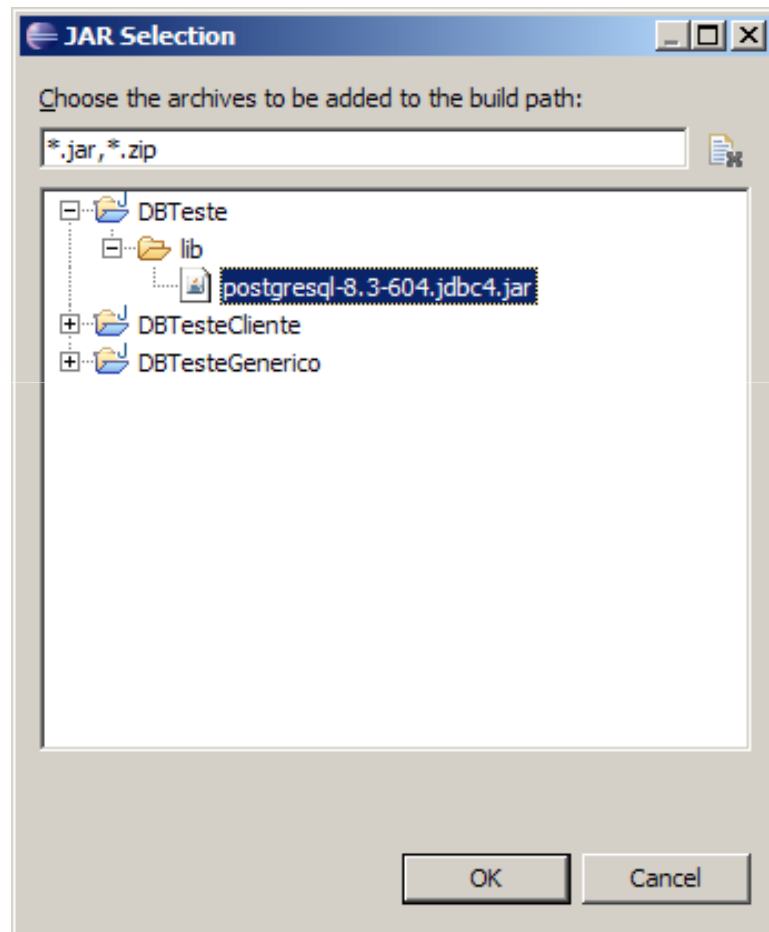
# Adicione o .jar no build path

- Vá nas propriedades do projeto (clique com o botão direito sobre o projeto e vá em Propriedades).
- Na opção Java Build Path, clique em Add JARs...e selecione o .jar do driver.
- A seguir em passos...

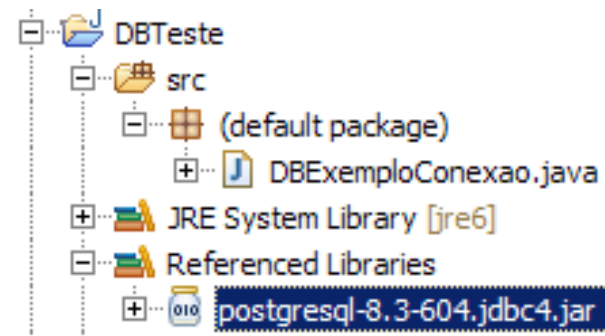
# Propriedades > Java Build Path



# Escolhendo o .jar do Driver



Então o driver aparecerá como uma Biblioteca Referenciada...



# Criando a classe de exemplo...

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```



# Carrega a classe org.postgresql.Driver na memória...

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```



# Abre uma conexão JDBC com o servidor local

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```



# Busca e mostra todos os registros da tabela clientes

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```



# Apagando clientes



```
System.out.println("Apagando todos os clientes...");  
s.executeUpdate("DELETE FROM clientes");
```



# Inserindo clientes



```
System.out.println("Inserindo clientes ...");
PreparedStatement p = c.prepareStatement(
    "INSERT INTO clientes (codigo, nome) VALUES (?, ?)");
p.setInt(1, 1);
p.setString(2, "Fulano");
p.execute();
p.setInt(1, 2);
p.setString(2, "Beltrano");
p.execute();
p.setInt(1, 3);
p.setString(2, "Ciclano");
p.execute();
```



# Alterando clientes



```
System.out.println("Alterando o cliente 1...");  
p = c.prepareStatement(  
    "UPDATE clientes SET nome = ? WHERE codigo = 1");  
p.setString(1, "Fulano de Tal");  
p.execute();
```



# Exercício

- Implementar a classe de exemplo.
- Listar os registros da tabela após cada operação.  
Ex.:
  - Listar todos os clientes
  - Apagar todos os clientes
  - Listar todos os clientes novamente
  - Inserir clientes
  - Listar todos os clientes novamente
  - Alterar clientes
  - Listar todos os clientes novamente
- Comparar com DBTeste.zip do site...